Exception Handling

Syntax Error

In [2]:
```
if 5 > 2
    print("5 is greater than 2")
```

```
  Cell In[2], line 1
    if 5 > 2
            ^
SyntaxError: expected ':'
```

In [5]:
```
print("Hello Students")
print("How are you all?)
```

```
  Cell In[5], line 2
    print("How are you all?)
          ^
SyntaxError: unterminated string literal (detected at line 2)
```

Runtime Error / Exception

In [8]:
```
a, b = 10, 0
print(a)
print(b)
c = a // b
print(c)
```

```
10
0
---------------------------------------------------------------------------
ZeroDivisionError                         Traceback (most recent call last)
Cell In[8], line 4
      2 print(a)
      3 print(b)
----> 4 c = a // b
      5 print(c)

ZeroDivisionError: integer division or modulo by zero
```

In [10]:
```
print(x)
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[10], line 1
----> 1 print(x)

NameError: name 'x' is not defined
```

In [12]:
```
print(10 + "20")
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[12], line 1
----> 1 print(10 + "20")

TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

In [14]:
```
num = int(input("Enter a number:"))
print(num)
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[14], line 1
----> 1 num = int(input("Enter a number:"))
      2 print(num)

ValueError: invalid literal for int() with base 10: 'abc'
```

In [16]:
```
l1 = [10,20,30,40,50]
print(l1[9])
```

```
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
Cell In[16], line 2
      1 l1 = [10,20,30,40,50]
----> 2 print(l1[9])

IndexError: list index out of range
```

In [18]:
```
d1 = {1:"One",2:"Two",3:"Three"}
```

In [20]:
```
d1[5]
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
Cell In[20], line 1
----> 1 d1[5]

KeyError: 5
```

In [22]:
```
import xyz
```

```
---------------------------------------------------------------------------
ModuleNotFoundError                       Traceback (most recent call last)
Cell In[22], line 1
----> 1 import xyz

ModuleNotFoundError: No module named 'xyz'
```

In [24]:
```
x = 25
x.append(50)
prin(x)
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[24], line 2
      1 x = 25
----> 2 x.append(50)
      3 prin(x)

AttributeError: 'int' object has no attribute 'append'
```

In [26]:
```
x = [25]
x.append(50)
prin(x)
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[26], line 3
      1 x = [25]
      2 x.append(50)
----> 3 prin(x)

NameError: name 'prin' is not defined
```

In [28]:
```
x = [25]
x.append(50)
print(x)
```

```
[25, 50]
```

In [30]:
```
assert 2 > 5
```

```
---------------------------------------------------------------------------
AssertionError                            Traceback (most recent call last)
Cell In[30], line 1
----> 1 assert 2 > 5

AssertionError:
```

In [32]:
```
assert 2 > 5 , "2 is not greater than 5"
```

```
---------------------------------------------------------------------------
AssertionError                            Traceback (most recent call last)
Cell In[32], line 1
----> 1 assert 2 > 5 , "2 is not greater than 5"

AssertionError: 2 is not greater than 5
```

In [34]:
```
a, b = 20, 5
assert a > b
print("code continued.")
```

```
code continued.
```

try except

In [37]:
```
a, b = 10, 5
try:
    c = a // b
    print(c)
except:
```

```
    print("b can not be 0")
print("End of the Program.")
```

```
2
End of the Program.
```

In [39]:
```
a, b = 10, 0
try:
    c = a // b
    print(c)
except:
    print("b can not be 0")
print("End of the Program.")
```

```
b can not be 0
End of the Program.
```

In [41]:
```
l1 = [10,20,30,40,50]
try:
    index = int(input("Enter index: "))
    print(l1[index])
except IndexError:
    print("Index out of bound.")
except ValueError:
    print("Invalid index is provided.")
print("End of the program.")
```

```
Invalid index is provided.
End of the program.
```

In [43]:
```
l1 = [10,20,30,40,50]
try:
    index = int(input("Enter index: "))
    print(l1[index])
except IndexError:
    print("Index out of bound.")
except ValueError:
    print("Invalid index is provided.")
print("End of the program.")
```

```
Index out of bound.
End of the program.
```

In [45]:
```
l1 = [10,20,30,40,50]
try:
    index = int(input("Enter index: "))
    print(l1[index])
except IndexError:
    print("Index out of bound.")
except ValueError:
    print("Invalid index is provided.")
print("End of the program.")
```

```
40
End of the program.
```

In [47]:
```
try:
    print("Hello World.")
    print("Good Morning.")
    print(10+56)
    print("Bye.")
except ZeroDivisionError:
```

```
    print("Some error.")
except:
    print("some error has occured.")
print("End of the code.")
```

```
Hello World.
Good Morning.
66
Bye.
End of the code.
```

In [49]:
```
l1 = [10,20,30,40,50]
try:
    index = int(input("Enter index: "))
    print(l1[index])
    print(10/0)
except Exception as err:
    print(err)
print("End of the program.")
```

```
40
division by zero
End of the program.
```

In [51]:
```
for x in ValueError.__mro__:
    print(x)
```

```
<class 'ValueError'>
<class 'Exception'>
<class 'BaseException'>
<class 'object'>
```

In [53]:
```
a, b = 10, 0
try:
    c = a // b
except Exception as err:
    print(err)
else:
    print("Result is :", c)
finally:
    print("End of the code.")
```

```
integer division or modulo by zero
End of the code.
```

raise

In [56]:
```
age = -23
try:
    if age <= 0:
        raise ValueError("Age can not be 0 or less.")
except ValueError as err:
    print(err)
else:
    print("Entered valid age.")
finally:
    print("Code Completed.")
```

```
Age can not be 0 or less.
Code Completed.
```

In [58]:
```
balance = 5000
try:
    amount = int(input("Enter the amount to be debited: "))
    if amount > balance:
        raise ValueError("Insufficient Balance.")
    if amount < 0:
        raise ValueError("Invalid Input.")
except ValueError as err:
    print(err)
else:
    balance -= amount
    print("Remaining Balance :",balance)
finally:
    print("Transaction Completed.")
```

```
Insufficient Balance.
Transaction Completed.
```

In [60]:
```
balance = 5000
try:
    amount = int(input("Enter the amount to be debited: "))
    if amount > balance:
        raise ValueError("Insufficient Balance.")
    if amount < 0:
        raise ValueError("Invalid Input.")
except ValueError as err:
    print(err)
else:
    balance -= amount
    print("Remaining Balance :",balance)
finally:
    print("Transaction Completed.")
```

```
Remaining Balance : 2500
Transaction Completed.
```

importance of finally block

In [63]:
```
a, b = 10, 5
try:
    c = a // b
except Exception as err:
    print(err)
else:
    print("Result is :", c)
print("End of the code.")
```

```
Result is : 2
End of the code.
```

In [69]:
```
def divide_(a,b):
    try:
        c = a // b
    except Exception as err:
        print(err)
    else:
        print("Division done successfully.")
        return c
    print("End of the code.")
```

In [71]:
```
divide_(10,2)
```

```
Division done successfully.
```

Out[71]: 5

In [73]:
```
def divide(a,b):
    try:
        c = a // b
    except Exception as err:
        print(err)
    else:
        print("Division done successfully.")
        return c
    finally:
        print("End of the code.")
```

In [75]:
```
divide(10,2)
```

```
Division done successfully.
End of the code.
```

Out[75]: 5

User Defined Exception

In [78]:
```
class NegativeAgeError(Exception):
    pass
```

In [80]:
```
try:
    age = int(input("Enter age : "))
    if age < 0:
        raise NegativeAgeError("Age can't be negative.")
except Exception as err:
    print(type(err).__name__,":", err)
else:
    print("Age is :",age)
finally:
    print("End.")
```

```
NegativeAgeError : Age can't be negative.
End.
```

In [82]:
```
class InsufficientBalanceError(Exception):
    pass
```

In [84]:
```
balance = 5000
try:
    amount = int(input("enter input"))
    if amount > balance:
        raise InsufficientBalanceError("Balance is insufficient.")
except Exception as err:
    print(type(err).__name__,":",err)
else:
    balance -= amount
    print("Remaining balance :",balance)
finally:
    print("Thank you for banking with us.")
```

```
InsufficientBalanceError : Balance is insufficient.
Thank you for banking with us.
```

In [86]:
```
balance = 5000
try:
    amount = int(input("enter input"))
    if amount > balance:
        raise InsufficientBalanceError("NA","Insufficient Balance")
except Exception as err:
    print(err)
else:
    balance -= amount
    print("Remaining balance :",balance)
finally:
    print("Thank you for banking with us.")
```

```
('NA', 'Insufficient Balance')
Thank you for banking with us.
```

OOP in Python

In [93]:
```
class Person:
    def __init__(self,name,roll_no): # constructor
        self.name = name
        self.roll_no = roll_no

    def __str__(self): #special method to return a string when an object is prin
        return str(self.roll_no)
```

In [95]:
```
p1 = Person("Darshan",101)
```

In [97]:
```
p1.name
```

Out[97]:
```
'Darshan'
```

In [99]:
```
p1.roll_no
```

Out[99]:
```
101
```

In [101…
```
print(p1)
```

```
101
```

How is the constructor of all exceptions?

In [ ]:
```
class Exception(BaseException):
    pass
```

In [ ]:
```
class BaseException(Object):
    def __init__(self,*args):
        self.args = args

    def __str__(self):
        if len(self.args) == 0:
            return ""
        elif len(self.args) == 1:
            return str(self.args[0])
```

```
        else:
            return str(self.args)
```

In [104…

```
try:
    print(10/0)
except ZeroDivisionError as err:
    print(type(err))
    print(err.args)
    print(err.args[0])
    print(err.__str__())
```

```
<class 'ZeroDivisionError'>
('division by zero',)
division by zero
division by zero
```